

# MRs for each scenarios

## MRPs/MRIPs

### The Entities-Updating MRIP:

The Entities-Updating MRIP refers to adding or removing the entities in the source input to construct the follow-up inputs.

#### **MRP<sub>FollowDirection</sub>:**

Follow the "direction" of the successful / failed scenario to build subsequent scenarios; the results should remain the same.

### The Entities-Reconfiguration MRIP:

The Entities-Reconfiguration MRIP refers to reconfiguring some properties of the entities in the source input to construct the follow-up inputs.

#### **MRP<sub>PropertyVariations</sub>:**

Create the variations of the property of the entities, and the results should remain the same.

### The Combining-Scenarios MRIP:

The Combining-Scenarios MRIP refers to combining the scenarios that are "independent" of each other in the source input to construct the follow-up inputs.

#### **MRP<sub>CombineScenarios</sub>:**

If two scenarios are independent of each other and the system outputs the same result, then by combining the two scenarios, the system output should remain unchanged.

# Scenarios

## FollowLeadingVehicle / FollowLeadingVehicleWithObstacle

The scenario realizes a common driving behavior, in which the user-controlled ego vehicle follows a leading car driving down a given road in Town01. At some point the leading car slows down and finally stops. The ego vehicle has to react accordingly (i.e. slow down too) to avoid a collision. The scenario ends either via a timeout, or if the ego vehicle stopped close enough to the leading vehicle

### MRP<sub>FollowDirection</sub>:

- Add another vehicle (vehicle 2) in front of the leading vehicle (vehicle 1) with the same initial speed. When vehicle 1 slows down, vehicle 2 slows down simultaneously but with lower deceleration, and the ego vehicle should behave the same as the original scenario.

### MRP<sub>PropertyVariations</sub>:

- **Change type:** change the type of the leading vehicle from car to other types of vehicles (e.g. truck, bicycle, etc.) should not affect the distance change between the ego vehicle and the leading vehicle.
- 

### MRP<sub>CombineScenarios</sub>:

- Scenario 1: there is a vehicle (vehicle 2) driving in the left lane of the leading vehicle (vehicle 1). The ego vehicle should behave the same as the original scenario.
- Scenario 2: there is a vehicle (vehicle 3) driving in the right lane of the leading vehicle with the opposite direction (vehicle 1). The ego vehicle should behave the same as the original scenario.
- Scenario 1 + 2: there are two vehicles (vehicle 2 and vehicle 3) driving in the left and right lane of the leading vehicle (vehicle 1). The ego vehicle should behave the same as the original scenario.

## VehicleTurningRight / VehicleTurningLeft

In this scenario the ego vehicle takes a right turn from an intersection where a cyclist suddenly drives into the way of the ego vehicle, which has to stop accordingly. After some time, the cyclist clears the road, such that ego vehicle can continue driving.

## **MRPFollowDirection:**

- Add other obstacles behind the cyclist, such as a pedestrian, a vehicle, etc. The ego vehicle should behave the same as the original scenario.
- Decrease the speed of the cyclist. The ego vehicle should behave the same as the original scenario.
- Change the direction of the cyclist (from left to right). The ego vehicle should behave the same as the original scenario.

## **MRPPropertyVariations:**

- Change type of the cyclist (from bicycle to other types of vehicles). The ego vehicle should behave the same as the original scenario.
- Adjust the initial position of the cyclist (little adjustment). The ego vehicle should behave the same as the original scenario.

## **MRPCombineScenarios:**

- Scenario 1: the source scenario
- Scenario 2: change the direction of the cyclist (from left to right)
- Scenario 1 + 2: two cyclists driving towards each other, and the ego vehicle should behave the same as the original scenario.

# **OppositeVehicleRunningRedLight**

(OppositeVehicleRunningRedLight\_3)

In this scenario an illegal behavior at an intersection is tested. An other vehicle waits at an intersection, but illegally runs a red traffic light. The approaching ego vehicle has to handle this situation correctly, i.e. despite of a green traffic light, it has to stop and wait until the intersection is clear again. Afterwards, it should continue driving.

## **MRPFollowDirection:**

- Decrease the speed of the other vehicle. The ego vehicle should behave the same as the original scenario.
- Add multiple vehicles running red light. The ego vehicle should behave the same as the original scenario.
- Change the speed of the other vehicle to make the ego vehicle hit the other vehicle. Then add other obstacles behind the other vehicle, and the ego vehicle should behave the same as the original scenario.

## **MRP<sub>PropertyVariations</sub>:**

- Change type of the other vehicle (from car to other types of vehicles/cyclists). The ego vehicle should behave the same as the original scenario.

## **MRP<sub>CombineScenarios</sub>:**

- Scenario 1: the source scenario
- Scenario 2: change the direction of the vehicle (from left to right)
- Scenario 1 + 2: two vehicles driving towards each other (both running red lights), and the ego vehicle should behave the same as the original scenario.

## **DynamicObjectCrossing/StationaryObjectCrossing**

This is similar to 'StationaryObjectCrossing', but with the difference that the cyclist is dynamic. It suddenly drives into the way of the ego vehicle, which has to stop accordingly. After some time, the cyclist will clear the road, such that the ego vehicle can continue driving.

## **MRP<sub>FollowDirection</sub>:**

- Add another obstacles behind the cyclist, such as a pedestrian, a vehicle, etc. The ego vehicle should behave the same as the original scenario.
- Decrease the speed of the cyclist. The ego vehicle should behave the same as the original scenario.
- Make the cyclist hit the ego vehicle. Then add another obstacles behind the cyclist, and the ego vehicle should behave the same as the original scenario.

## **MRP<sub>PropertyVariations</sub>:**

- Change type of the cyclist (from bicycle to other types of vehicles). The ego vehicle should behave the same as the original scenario.
- Change the starting position of the cyclist (little adjustment). The ego vehicle should behave the same as the original scenario.

## **MRP<sub>CombineScenarios</sub>:**

- Scenario 1: the source scenario
- Scenario 2: change the direction of the cyclist (from left to right)
- Scenario 1 + 2: two cyclists driving towards each other, and the ego vehicle should behave the same as the original scenario.

# NoSignalJunctionCrossing

This scenario tests negotiation between two vehicles crossing cross each other through a junction without signal. The ego vehicle is passing through a junction without traffic lights and encounters another vehicle passing across the junction. The ego vehicle has to avoid collision and navigate across the junction to succeed.

## MRPFollowDirection:

- Change type of the other vehicle (from car to other types of vehicles/cyclists). The ego vehicle should behave the same as the original scenario.
- Add another obstacles behind the other vehicle, such as a pedestrian, a vehicle, etc. The ego vehicle should behave the same as the original scenario.

## MRPPropertyVariations:

- Change the lane of the other vehicle. The ego vehicle should behave the same as the original scenario.
- Move the starting point of the other vehicle to closer. The ego vehicle should behave the same as the original scenario.

## MRPCombineScenarios:

- Scenario 1: the source scenario
- Scenario 2: Another vehicle turned left from opposite direction to enter the same direction as the ego vehicle. The ego vehicle should behave the same as the original scenario.
- Scenario 1 + 2: One vehicles turned left from opposite direction to enter the same direction as the ego vehicle, and the other vehicle drives as the source scenario. The ego vehicle should behave the same as the original scenario.

# SignalizedJunctionRightTurn/SignalizedJunctionLeftTurn

In this scenario right turn of hero actor without collision at signalized intersection is tested. Hero Vehicle is turning right in an urban area, at a signalized intersection and turns into the same direction of another vehicle crossing straight initially from a lateral direction.

## MRPFollowDirection:

- Change type of the other vehicle (from car to other types of vehicles/cyclists). The ego vehicle should behave the same as the original scenario.

- Decrease the speed of the other vehicle. The ego vehicle should behave the same as the original scenario.

### **MRPPropertyVariations:**

- Change the lane of the other vehicle. The ego vehicle should behave the same as the original scenario.

### **MRPCombineScenarios:**

- Scenario 1: the source scenario
- Scenario 2: Another vehicle turned left from opposite direction to enter the same direction as the ego vehicle. The ego vehicle should behave the same as the original scenario.
- Scenario 1 + 2: One vehicles turned left from opposite direction to enter the same direction as the ego vehicle, and the other vehicle drives as the source scenario. The ego vehicle should behave the same as the original scenario.